

Advanced Urban Visualization

Scalability, Performance & 3D Interaction · Part 1 & 2

Raz Saremi, Ph.D.

Tandon School of Engineering, NYU

Information Visualization

Learning Objectives

What you will master in this session.

01

Scale & Performance

Understand the challenges of visualizing massive urban datasets — millions of polygons + real-time IoT data — and maintain 60 FPS.

02

LOD & Culling

Master Level of Detail, frustum culling, occlusion culling, spatial indexing, and batching for 3D city renderers.

03

3D Interaction Design

Design and evaluate 3D navigation metaphors, selection techniques, multi-scale interaction, and collaborative VR/AR workflows.

The Challenge of Scale

Urban environments are data-dense by nature.

100K+

Buildings in a typical metropolitan model (Tokyo: ~850,000)

50M+

Polygons visible in a single street-level city view

60 fps

Target frame rate — each frame must complete in 16.67 ms

IoT

Real-time sensor streams: traffic, air quality, people density, energy

The Frame Budget

16.67 ms is the total budget for geometry, shadows, physics, streaming, and UI — all at once.

The Data Problem

A single BIM export of one block can exceed 500,000 polygons. Without optimization, 100 blocks = 50 M triangles before a single shadow is cast.

Key Insight: *Urban visualization is a real-time data-engineering problem as much as a graphics problem.*

Defining Performance Bottlenecks

Diagnosing where your 16 ms budget is being spent.

CPU Bound	GPU Bound	Memory Bound
- Game logic, AI, physics - Draw call preparation - Culling decisions - Data parsing / streaming	- Vertex transform & clipping - Fragment / pixel shading - Shadow map passes - Post-processing FX	- Texture streaming (VRAM) - Mesh buffer uploads - CPU ↔ GPU bandwidth - Point-cloud buffer size

Diagnostic: If GPU is 100% + CPU idle → GPU bound. If draw calls > 5K + CPU 100% → CPU bound. If VRAM full + stutters on movement → Memory bound.

Level of Detail (LOD) Basics

Reducing geometric complexity as objects move further from the camera.

LOD Level	Polygon Count	Distance	Detail Description
LOD 0 — Full Detail	50,000 △	< 100 m	Bricks, window frames, cornices, rooftop AC units
LOD 1 — High	15,000 △	100–300 m	Facade geometry, window grid — no micro-detail
LOD 2 — Medium	3,000 △	300–700 m	Building silhouette with rough surface subdivision
LOD 3 — Box	12 △	> 700 m	Simple extruded box — 2–4 pixels tall on screen

Key Principle: *The human visual system cannot resolve fine detail at distance — LOD exploits this perceptual limit to save GPU budget.*

Discrete vs. Continuous LOD

Two strategies for managing geometry transitions.

Discrete LOD

Mechanism: Separate pre-baked meshes at LOD 0–3. Swapped instantly when distance crosses a threshold. $O(1)$ runtime cost.

Pro: Extremely fast lookup. Zero per-frame computation overhead.

Con: Visible "pop" artifact at threshold boundary.

Mitigation: LOD cross-fade dithering; place thresholds where building subtends very few pixels.

Continuous (Geomorphing) LOD

Mechanism: Vertex positions interpolate smoothly between LOD levels via a morph factor ($0 \rightarrow 1$) based on distance.

Pro: No visible pop — seamless transitions. Essential for hero buildings under direct camera focus.

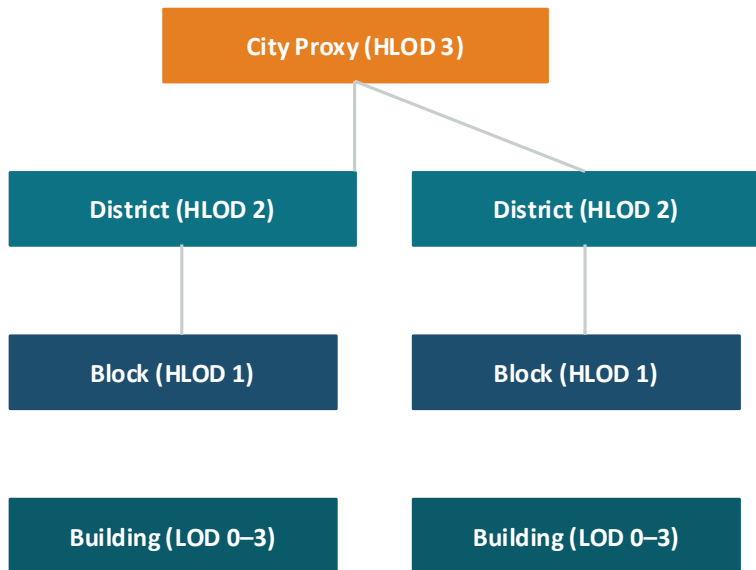
Con: ~25% more vertex shader instructions per vertex. Impractical for all 100K city buildings.

UE5 Nanite: Virtualized geometry — clusters auto-selected per pixel on GPU. Eliminates manual LOD authoring.

Hierarchical Level of Detail (HLOD)

Grouping city blocks into single proxy meshes for distant views.

HLOD Hierarchy



Concept

Merge many buildings into one combined proxy mesh when viewed far away. A whole downtown block = 1 draw call.

Performance

Reduces draw calls from 500,000 (individual buildings) to ~500 (block proxies) at city-bird-eye altitude.

Baking Process

Offline spatial clustering → merge geometry → atlas textures. Zero runtime cost; stored as pre-built static asset.

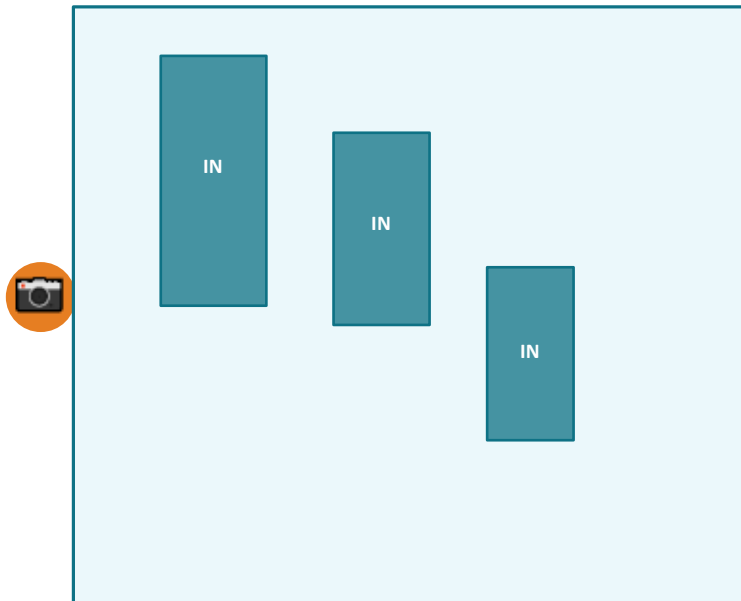
Industry Use

Unreal Engine HLOD, CesiumJS 3D Tiles, Bentley iTwin all use hierarchical proxy systems for city-scale streaming.

Visibility Culling: Frustum Culling

Only render what is within the camera's field of view.

Max Visible Fraction at 60° FOV



Frustum (View Pyramid)

What Gets Culled

Any object whose AABB is entirely outside all 6 frustum planes. Zero GPU cost — never enters the pipeline.

AABB Testing

6 dot-product plane tests per building AABB. With a quadtree: only test spatial branches overlapping the frustum. $O(\log n)$.

Performance Impact

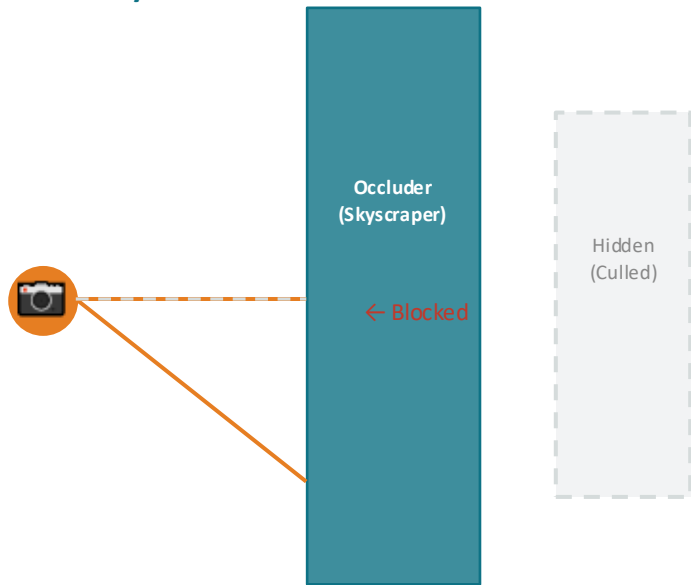
At street level, 85–95% of city geometry is outside the frustum. This is the cheapest, most impactful optimization — apply first.

Exercise: A 10×10 city grid, 60° FOV at street level. Estimate: how many of the 100 buildings are inside the frustum? (Answer: ~8–12)

Visibility Culling: Occlusion Culling

Don't render buildings hidden behind larger buildings.

Urban Canyon — Occlusion Scenario



HZB (Hierarchical Z-Buffer)

GPU builds a depth-buffer mipmap. Each object tests its projected min-depth against the HZB — rejected early if fully occluded. All on GPU.

Portal / Cell Systems

City divided into cells connected by portals. Visibility traced through visible portal chains only — ideal for interior spaces.

Urban Canyon Savings

Dense street canyons: foreground skyscrapers occlude 70–85% of background geometry after frustum culling. Compound savings are dramatic.

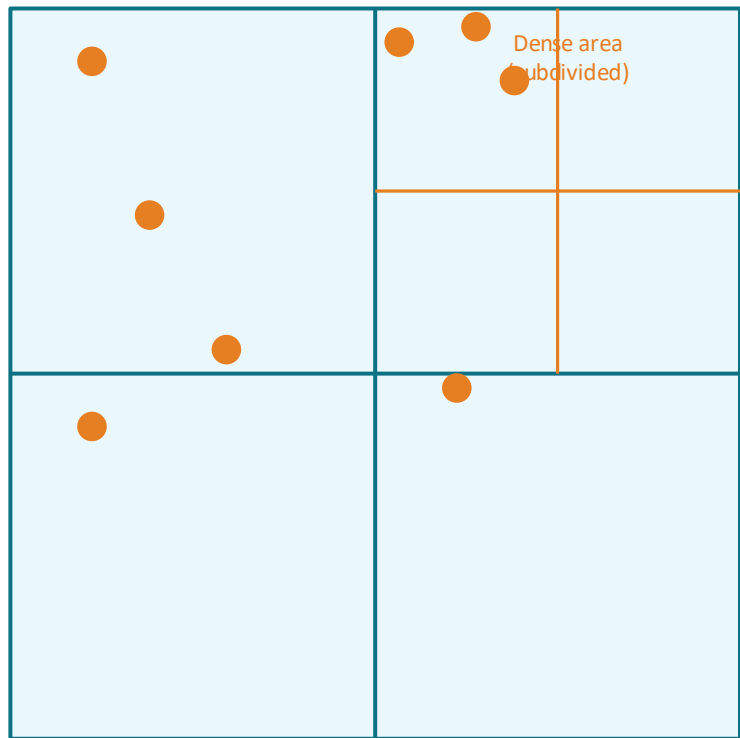
Limitation

Dynamic objects (cars, crowds) cannot be used as reliable occluders. Only static buildings serve as occluders in most implementations.

Spatial Indexing: Quadtrees & Octrees

Organizing city data for $O(\log n)$ geographic queries.

Max Possible Edges = $n(n-1) / 2$



Structure	Dimensions	Query: $O(\dots)$
Quadtree	2D (x,y)	$\log n + k$
Octree	3D (x,y,z)	$\log n + k$
R-Tree	nD	$\log n + k$
KD-Tree	nD	$\sqrt{n}$ (avg)

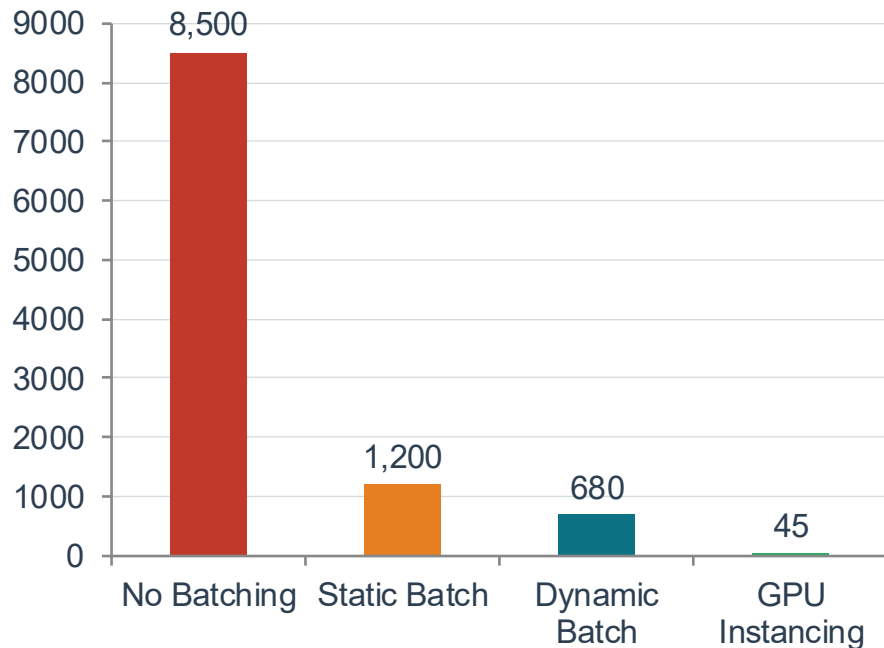
Industry Tools

PostGIS (R-Tree), Mapbox GL (quad-tiles), Rbush (R-Tree JS), ESRI ArcGIS spatial index, CesiumJS octree.

Query performance: 100 K buildings — linear scan = 100,000 tests; octree = ~17 traversal steps.

Geometry Batching

Minimizing draw calls by combining geometry.



Draw Calls: Urban Street Scene (1,000 objects)

Static Batching

Pre-merge immovable objects (benches, lamps) sharing the same material at load time → one large mesh. Zero runtime overhead.

Dynamic Batching

Runtime merge of small moving objects sharing one material. Unity limit: 300 vertices per object. CPU cost per frame.

GPU Instancing

1 mesh + 1 position/color buffer = 10,000 identical objects in 1 draw call. Target < 500 draw calls/frame for interactive city apps.

Key Requirement

Batching only works between objects sharing the same material — this is why texture atlasing (slide 13) is a prerequisite.

Instanced Rendering

One mesh description — 10,000 placements — one draw call.

10,000×

Instances drawn per draw call (trees, lamps, bollards, windows)

~0.1 ms

GPU time for 10,000 instanced trees on a mid-range GPU

80 bytes

Per-instance data: position (12) + rotation (16) + scale (12) + color (16) + misc (24)

How Instanced Rendering Works

1. Upload ONE mesh to VRAM once.
2. Upload instance buffer: position, rotation, scale, color per instance.
3. Issue `DrawInstancedPrimitive(mesh, buffer, count=10000)` — 1 draw call.
4. GPU spawns 10,000 transformed copies in parallel.

Common Instance Types in Urban Scenes

Street Lamps

Bollards & Benches

Facade Windows

Crowd Agents

Road Signs

Tree Canopy

API Examples

WebGL: `gl.drawElementsInstanced()`
Unity: `Graphics.DrawMeshInstanced()`
Three.js: `new THREE.InstancedMesh()`

Texture Atlasing & Compression

Maximising GPU memory efficiency for city surface materials.

Texture Atlas (4096 × 4096)



GPU Compression Formats

Format	Ratio	Quality	Platform
BC7 / DXTC	6:1	☆☆☆ ☆☆	PC / Console
ASTC	25:1	☆☆☆ ☆	Mobile / ARM
ETC2	6:1	☆☆☆	Android
Uncompressed	—	☆☆☆ ☆☆	All (max VRAM)

Memory Saving Example

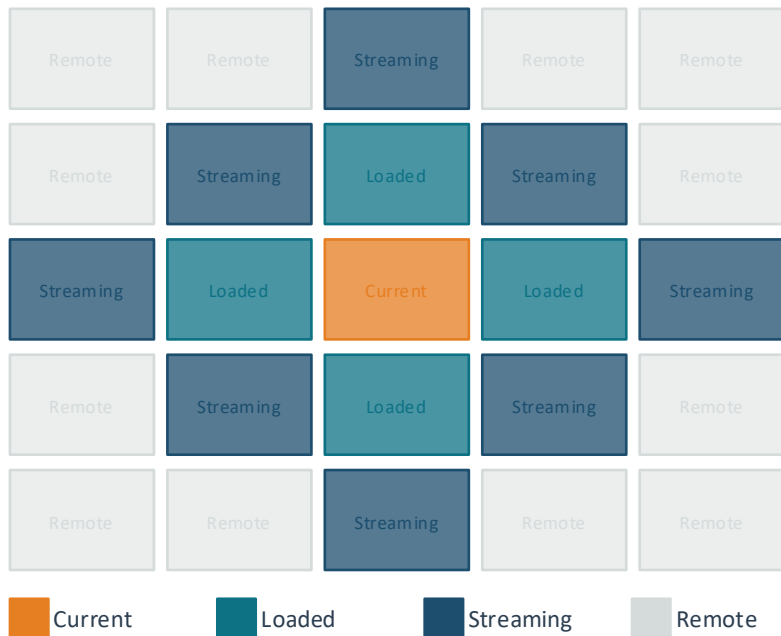
Uncompressed 4K RGBA atlas: 67 MB → BC7 with mipmaps: ~15 MB. 50 atlases: 3.35 GB → 750 MB — fits in consumer GPU VRAM.

Atlas benefit: 1 texture bind for all buildings sharing the atlas vs. N separate binds — enables aggressive batching.

Data Streaming Strategies

Loading the next city block before the user arrives.

Tile State Map — City Grid



Predictive Prefetch

Track camera velocity → preload tiles in movement direction 2–3 seconds ahead. Used by Google Maps and CesiumJS.

Priority Queue

Closer tiles load first. LOD 0 tiles get priority over LOD 2. Background threads handle disk/network I/O asynchronously.

3D Tiles (OGC std.)

Hierarchical tile manifest (tileset.json) with geometric error per tile. Supports additive and replacement refinement modes.

Cache Eviction

LRU eviction: tiles beyond threshold are removed from VRAM when budget is tight. Minimum tiles always cover current view.

Point Cloud Visualization

Rendering millions of LiDAR points in real time.

What is LiDAR?

Laser pulses measure precise distances. A city scan produces 100M–1B points each with XYZ + intensity + RGB classification.

Rendering Challenge

No triangles — points rendered as billboarded splats via vertex shaders. At 20 bytes/point: 1B points = 20 GB. Needs streaming.

Octree Paging

Point cloud partitioned into an octree. Leaf nodes streamed on demand by camera proximity. Tools: Potree, PDAL, Entwine.

Point Splatting

Each point → small oriented disc perpendicular to camera. EWA (Elliptical Weighted Average) splatting provides anti-aliasing.

Point Budget

User sets max renderable points (e.g., 2M). Potree selects octree nodes to fill budget by projected screen size — $O(\log n)$.

100M–
1B

Points per city LiDAR scan

20 bytes

Per-point storage (XYZ + intensity + RGB)

Potree

Leading browser-based point cloud renderer (WebGL + octree streaming)

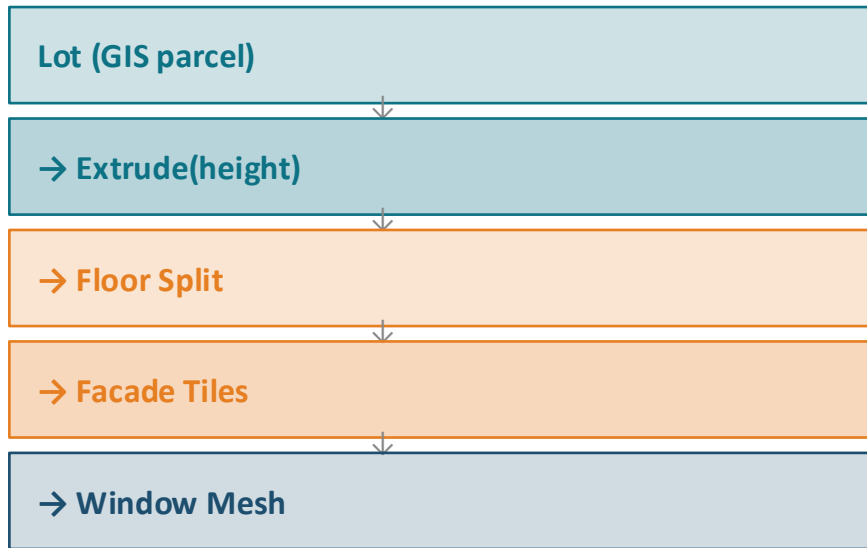
Industry Applications

Autodesk Recap • ESRI ArcGIS Point Cloud • Mapbox HD Roads • HERE Maps • Autonomous vehicle training data

Procedural Modeling for Performance

Generating building facades on-the-fly using grammar rules.

CGA Shape Grammar — Rule Chain (CityEngine)



Storage Savings

CGA rule set: ~5 KB vs. a single high-quality mesh: ~15 MB → 3,000× reduction. An entire city stored as rules.

L-Systems (Roads)

Parametric grammars grow road networks from a seed rule. Inputs: density field, slope, highway attractors. Output: geometry on demand.

CityEngine (ESRI)

Industry standard CGA tool. Ingests GIS parcel data (shapefile/GDB) + assigns rule sets by land use type → generates city procedurally.

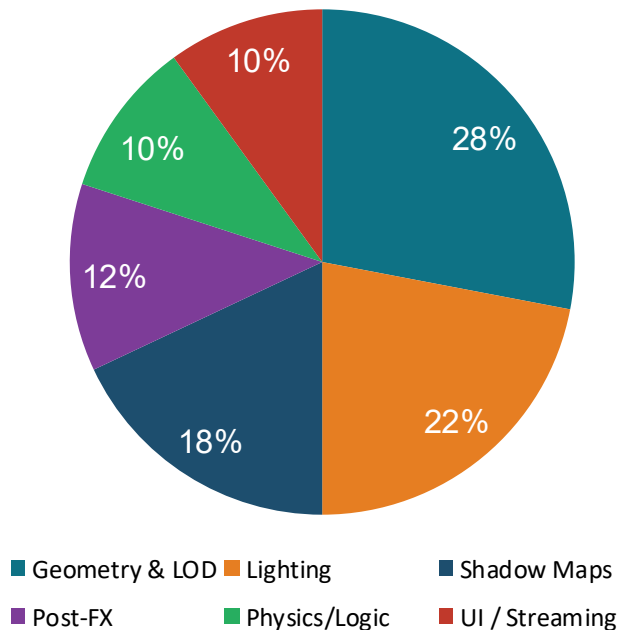
Use Cases

Urban planning simulation, VFX (The Dark Knight, Game of Thrones cityscapes), autonomous vehicle training data generation.

Exercise: Design a 3-rule grammar for a NYC brownstone: Lot → Base → Floors → Cornice. What parameters would you expose?

Balancing Fidelity vs. Performance

Allocating the 16.67 ms frame budget across rendering systems.



16.67 ms Frame Budget Allocation

Quality Presets

Preset	Geometry	Shadows	FPS
Ultra	High	4096px	24
High (Desktop)	Med	2048px	60
Medium (Web)	Low	1024px	60
Low (Mobile)	Box	None	30

Adaptive Quality

UE5 / Unity HDRP auto-scale quality at runtime: FPS drops → reduce shadow res → then lighting → then geometry, in priority order.

Client-Side vs. Server-Side Rendering

Where does the GPU work happen?

Client-Side Rendering

Device GPU: User's phone / laptop / desktop.

Delivery: WebGL / WebGPU for browser; native for desktop apps.

Latency: Zero network round-trip for interaction — immediate response.

Limit: Capped by device capability — mobile GPU $\neq$ RTX 4090.

Best for: Desktop enterprise tools, kiosks, web dashboards.

Server-Side (Pixel Streaming)

Device GPU: Cloud server (A100, RTX 4090) — full fidelity.

Delivery: H.264/H.265 video stream via WebRTC to thin client browser.

Latency: ~50–150 ms input-to-display round trip — acceptable for planning.

Quality: Full ray tracing, Nanite, Lumen — cinematic quality on mobile.

Best for: Mobile access to complex digital twins, public kiosks.

Industry: Epic Pixel Streaming (UE5) · Azure Remote Rendering (ARR / HoloLens 2) · NVIDIA CloudXR

Case Study: Unreal Engine 5

Nanite, Lumen, World Partition & Cesium — next-generation urban rendering.

Nanite — Virtualized Geometry

GPU selects only clusters projecting ≥ 1 pixel. Billions of polygons at native resolution. No manual LOD authoring for static geometry.

Lumen — Dynamic Global Illumination

Real-time GI: sunlight bouncing off glass towers, indirect light in street canyons. No lightmap baking. Time-of-day changes instantly.

World Partition

City divided into streaming cells, loaded by proximity. Supports maps $> 4,000 \text{ km}^2$ in a single UE5 project without manual streaming zones.

Cesium for Unreal

Streams Google 3D Photorealistic Tiles, Bing Maps, OSM directly into UE5 via the OGC 3D Tiles standard — real cities as interactive environments.

"The Matrix Awakens" (2021): UE5 demo — real-time city of 35,000 unique buildings using Nanite + Lumen at 30fps on console.

Performance Summary & Checkpoint

Consolidate Part 1 before moving to interaction.

LOD & HLOD

Reduce geometry per-object and per-city-block with distance.

Culling

Frustum + Occlusion eliminate unseen geometry before the GPU.

Spatial Index

Quadtree/Octree for $O(\log n)$ city queries and culling.

Batching

Minimize draw calls; GPU Instancing for repeated objects.

Streaming

Async tile loading; OGC 3D Tiles standard for geospatial.

Procedural

Grammar rules generate geometry on demand — no storage cost.

Quiz

Q1: A building at 1km uses LOD 3. Camera moves to 200m — which LOD triggers and why? Q2: Name two techniques that each cut draw calls by > 90%. Q3: Why does occlusion culling save more in a dense urban canyon vs. open terrain?

PART 2

Interaction Techniques in 3D Environments

From looking → doing: Perception, Decision, Action.

The Navigation Challenge

Scale varies from a single room to an entire metropolitan region.

Scale Range — 4 Orders of Magnitude

1:1	Room / Floor	Furniture, fixtures, structural columns
1:100	Building	Facades, floor plans, MEP systems
1:1,000	Block	Buildings, streets, open space
1:10,000	District	Zoning, neighborhoods, infrastructure
1:100,000	City / Region	Topography, districts, transit corridors

Key Insight: No single interaction metaphor handles all five scale levels gracefully — hybrid systems are necessary.

6 Degrees of Freedom (6DOF)

 Translate X	Left / Right
 Translate Y	Up / Down
 Translate Z	Forward / Back
 Pitch	Nod — look up/down
 Yaw	Turn — look left/right
 Roll	Tilt — almost always disabled

Classic Metaphors: The "World in Hand"

Rotating and scaling the city as if it were a physical model on a table.

World in Hand

Concept

City as miniature model. User grabs/rotates the world — camera stays fixed; world transforms.

Gestures

Drag to orbit (arcball). Pinch/zoom to scale. Two-finger translate for pan.

Strengths

Excellent for overview tasks, urban planning, public presentations. Low disorientation risk.

Weaknesses

Breaks immersion. Fails at 1:1 scale — no 'sense of being there.'
Limits interior inspection.

Arcball Rotation

Algorithm

Mouse position projected onto invisible sphere. Rotation between old/new intersection points defines transform. Quaternion-based — no gimbal lock.

Touch Mapping

Two-finger angle change → rotation. Finger distance change → scale. Both computed simultaneously.

Industry Tools

Google Maps 3D, Apple Maps 3D flyover, ArcGIS Scene Viewer, Cesium Viewer, CityEngine viewport.

World-in-Hand is the default metaphor for most web-based urban visualization tools — familiar, low learning curve, touch-friendly.

Classic Metaphors: Flying & Walking

First-person vs. drone-view urban navigation.



Drone / Fly-Through

- Full 6DOF: WASD + mouse look (desktop) or joystick (VR)
- Ideal for fly-throughs, real estate, infrastructure inspection
- Decoupled mode: body pans on horizontal plane, look direction is independent — prevents 'going underground'
- Collision option: gentle push-back when approaching building geometry
- Tools: Unreal Fly mode, CesiumJS camera.flyTo(), Google Earth 3D Tour



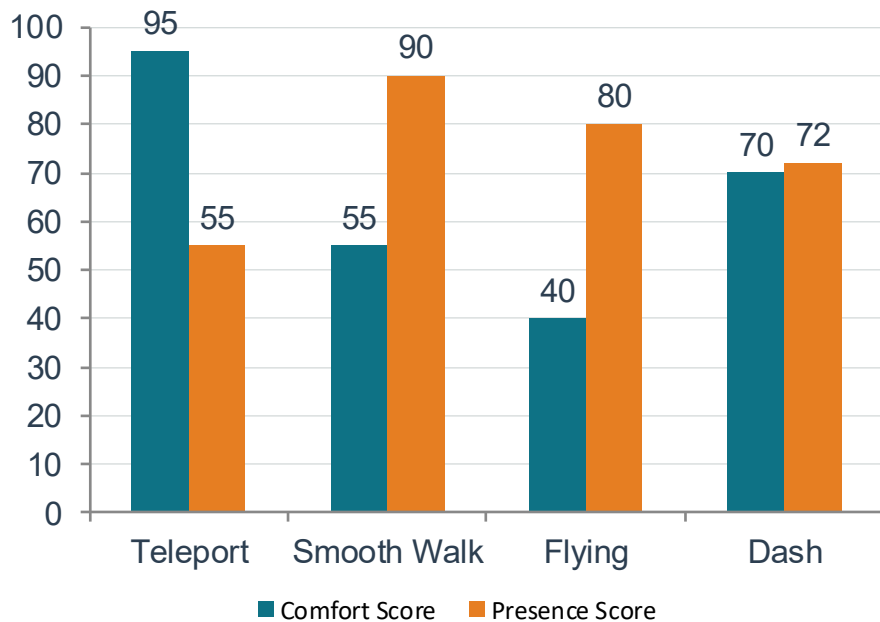
Walk-Through

- Camera height locked at ~1.7m (eye level) above terrain
- Downward raycast each frame maintains correct height on uneven terrain
- Collision enforced via capsule collider (~0.4m radius) against building geometry
- Challenge: narrow urban canyons require careful CCD (Continuous Collision Detection) tuning
- Tools: UE5 Character Controller, Unity NavMesh, Matterport 3D Tours

Best practice: offer both modes with keyboard shortcut to toggle — different tasks require different perspectives.

Teleportation vs. Smooth Locomotion

Addressing motion sickness (Simulator Sickness) in VR urban environments.



Comfort vs. Presence (score out of 100)

Teleportation

Point → instantly appear. No continuous motion → near-zero sickness. Lower presence. Best for city-scale jumps.

Smooth Locomotion

Thumbstick moves continuously. Highest presence but triggers sickness in ~40% of VR users. Mitigation: vignette.

Vignetting

Progressive black oval narrows FOV during movement — reduces peripheral optic flow that causes vestibular conflict. ~40% sickness reduction.

Hybrid Best Practice

Teleport between city blocks; smooth-walk within a building. Scale-appropriate locomotion — offer toggle for user preference.

Camera Constraints & Guidance

Preventing navigation errors while preserving exploration freedom.

Ground Clipping Prevention

Downward raycast from camera each frame. If hit distance < 1.7 m, set `camera.y = terrain_height + 1.7` m. Handles terrain transitions with interpolation.

Altitude Limits

Max height cap keeps camera within scene AABB — prevents 'lost in space' errors in web-based city apps with non-expert users.

Pitch Clamping

Pitch locked to $\pm 85^\circ$ to prevent gimbal lock at $\pm 90^\circ$ and disorienting upside-down views. Yaw remains unrestricted (full 360°).

Building Penetration Guard

Broad-phase AABB overlap $\rightarrow$ narrow-phase penetration depth + normal $\rightarrow$ push camera out along surface normal. CCD prevents fast-movement tunneling.

Path Smoothing (Splines)

Camera waypoints interpolated with Catmull-Rom splines. Prevents jerky guided-tour motion. C1 continuity = smooth velocity across waypoints.

Third-Person Arm Safety

Camera arm swept using capsule test. If building intersects arm, auto-zoom in along arm vector — maintains clear view of target.

Wayfinding Aids

Keeping users oriented in the virtual city.

Lynch's 5 Mental Map Elements (The Image of the City, 1960)

Paths

Streets, transit — channels of movement

Edges

Coastlines, highways — linear boundaries

Districts

Neighborhoods — mentally entered regions

Nodes

Plazas, intersections — focal points

Landmarks

Towers, monuments — external reference points

Minimap

Top-down overview in corner showing position + heading. North-up or camera-relative modes. Clickable for fast repositioning. Scale updates with zoom.

Compass Rose

Always-visible N/S/E/W. Critical for aligning mental map to real geography. Animates to reflect camera pitch + yaw simultaneously.

Breadcrumb Trail

Ghost path of recent camera positions. Auto-fades after 60 s. Effective for retracing short routes; disable for long sessions to avoid clutter.

Billboarded Labels

District names + building POI icons float in 3D space, always facing camera. Fade with distance. Depth-tested — hidden when building is in front.

Selection Techniques: Raycasting

Pointing at a building to retrieve its data.

Picking Pipeline: Screen → World

1. Mouse (px,py)	Screen coordinates	→
2. NDC coords	$(2x/W-1, 1-2y/H)$	→
3. Clip space	$\text{vec4}(\text{ndcX}, \text{ndcY}, -1, 1)$	→
4. View space	$\text{inverse}(\text{proj}) \times \text{clip}$	→
5. World ray	$\text{inverse}(\text{view}) \times \text{view}$	→
6. BVH test	$O(\log n)$ AABB tests	✓

BVH Optimization

Bounding Volume Hierarchy turns $O(n)$ ray-AABB tests into $O(\log n)$. 100K buildings: brute force = 100,000 tests; BVH = ~17 traversal steps.

Fat Finger Problem

Small screen targets = large 3D ambiguity. Solutions: snap to nearest centroid within cone, expand hit box, or disambiguation popup menu.

Data Retrieval on Hit

Building ID → async database query for attributes (floors, year, use, owner, energy, tenant data). Displayed in a diegetic floating panel.

Industry Examples

CesiumJS `ScreenSpaceEventHandler`, ArcGIS `SceneView.pick()`, Three.js `Raycaster` — all implement the same screen-to-world pipeline.

Selection Techniques: Cone & Volume Selection

Selecting multiple assets within a 3D volume.

Technique	How It Works	Urban Planning Use Case	Cost
Cone Cast	3D cone from camera; select objects whose centroid falls within cone aperture angle	Select all buildings facing a street corridor	$O(\log n)$
Sphere Select	3D sphere at cursor; user drags radius; query spatial index within sphere	All buildings within 500m of proposed metro station	$O(\log n + k)$
Lasso + Frustum	2D screen lasso → extruded into 3D frustum; select all AABB-intersecting objects	Select all buildings in a flood zone boundary	$O(\log n + k)$
Volume Box	User defines axis-aligned 3D box via gizmo; batch-select all enclosed objects	Select all buildings in a city block for zoning update	$O(\log n + k)$

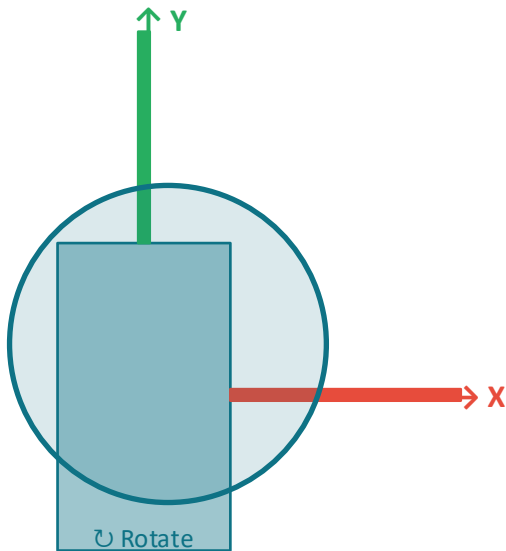
Discussion

After selecting 200 buildings: what operations might a planner want? → Aggregate stats (total FAR, population), apply zoning change, export to GIS, run shadow analysis, bulk attribute edit.

Manipulation: 3D Gizmos

Translating, rotating, and scaling objects in an urban planning context.

3D Transform Gizmo



What is a Gizmo?

In-world handle widget overlaid on selected object. Three axes (XYZ) control translate, rotate, or scale. Mode switched via toolbar icon.

Urban Planning Application

Move proposed building to test setback requirements. Rotate to optimise solar exposure. Scale to test different floor-area ratios (FAR).

Snap Constraints

Grid snap (align to parcel lines), angle snap (90°/45°/30°), surface snap (terrain-following for hillside sites). Essential for precise siting.

Gimbal Lock Solution

Internal quaternion representation eliminates gimbal lock. Display Euler angles in UI for human readability; compute with quaternions internally.

Industry Tools

Autodesk Forma, Rhino 3D, Unreal Editor, Bentley MicroStation — all use this paradigm for building placement and massing studies.

Multi-Scale Interaction

Seamlessly zooming from 1:1 street view to 1:100,000 regional view.

Scale	Level	Content	Navigation Metaphor	Selection
1:1	Room / Floor	Furniture, fixtures, structural columns	Walk / Grab	Click fixture
1:100	Building	Facades, floor plans, MEP systems	Orbit / Inspect	Click panel
1:1,000	Block	Buildings, streets, open space	Fly / Pan	Raycast + cone
1:10,000	District	Zoning, neighborhoods, land use	World-in-Hand	Lasso / radius
1:100K	City / Region	Infrastructure, topography, corridors	Map Pan / Zoom	Area select

Transition design: animated zoom (Catmull-Rom path), instantaneous teleport + cross-fade, or 'magic carpet' zoom centered on cursor position.

UI in 3D Space: Diegetic vs. Non-Diegetic

Floating labels (Billboarding) vs. Screen-space overlays.

Diegetic UI (World-Space)

Definition: Exists inside the 3D world — part of the scene.

Examples: Billboarded building labels, street-level POI markers, 3D panels floating near selected building.

Strengths: High immersion, contextually placed, no screen real estate cost.

Weaknesses: Readability affected by perspective, size, and distance. Occlusion by geometry can hide critical info.

Non-Diegetic UI (Screen-Space)

Definition: Overlaid on the screen — independent of 3D world depth.

Examples: 2D charts, filter panels, legend, minimap, toolbars, KPI dashboards.

Strengths: Always readable, precise interaction (mouse), complex data display.

Weaknesses: Occludes 3D content when overused. Disconnects data from spatial context — user must mentally re-link.

Best practice: Hybrid — diegetic labels for spatial identity; non-diegetic panels for data-heavy analysis (charts, filters, timelines).

Collaborative Urban Visualization

Multi-user interaction in shared virtual city spaces.

Shared World State

All users see the same model in real time. Sync via CRDT or WebSocket with conflict resolution (last-writer-wins or OT — Operational Transform).

Spatial Audio

Voice from avatar position: volume decreases with distance. Spatial audio enables natural sub-group formation — like a physical site visit.

NVIDIA Omniverse (USD)

Multi-user city model editing via Universal Scene Description (USD) opinion layers — non-destructive, version-controlled, mergeable per author.

Avatar Representation

Colored avatar + name label above. Head direction communicates gaze; controller ray shows pointing gesture — 'this building here' without ambiguity.

Annotations & Markup

Virtual laser pointer: visible ray from controller, seen by all collaborators. Freehand 3D strokes + sticky notes anchored to building coordinates.

Other Industry Platforms

Microsoft Mesh, Autodesk Tandem, Bentley ProjectWise, Epic Pixel Streaming — all support multi-user urban digital twin collaboration.

Annotations and Markups

Methods for stakeholders to mark up a 3D city model for planning feedback.

Annotation Type	Implementation	Storage	Integration
Sticky Notes	World-anchor + text + author + timestamp	DB: annotationID + coordinates + version	Jira / Linear ticket auto-created
3D Markup Lines	Controller → world-space polyline samples at 60fps	Vertex list in building-local coords (LOD-safe)	Review layer in Autodesk ACC
Dimension Tags	Snap to edge/corner → compute 3D distance	Tag ID + two anchor points + computed distance	Code compliance check: setbacks, heights
Volume Markup	3D box gizmo → encloses a spatial issue zone	AABB min/max + status (open/resolved/deferred)	Planning workflow: assign to engineer

Exercise:

Design an annotation workflow for a community meeting: residents, city planners, and engineers mark up a proposed light-rail corridor. What roles, permission levels, and issue categories would you define?

Haptic Feedback and Audio Cues

Using spatial audio and haptics to improve scale perception and presence.

Haptic on Selection

Short vibration pulse when raycast hits a building. Confirms action without visual lookup — reduces cognitive load in complex scenes.

HRTF Spatial Audio

Head-Related Transfer Function filter: models how ear shape creates directional cues. Enables truly 3D audio positioning — front/back/above.

Audio Occlusion

Sound muffled when source is around a corner or behind a building. Steam Audio provides real-time convolution — doorways filter sound correctly.

Haptic Resistance

Force feedback (VR gloves) simulates wall collision. Prevents inadvertent penetration of building geometry during close inspection.

Room Impulse Response

Reverb tail encodes space geometry. Narrow alley: long parallel-reflection reverb. Open plaza: short diffuse reverb. Enhances scale perception.

Industry Tools

Unity Audio, Unreal MetaSounds, Oculus Audio SDK, Steam Audio — all support real-time HRTF + reverb for virtual urban acoustic environments.

AR Interaction in the Real City

Aligning digital twins with physical buildings.

Spatial Anchoring

GPS ($\pm 3\text{m}$) + IMU + Visual SLAM (Simultaneous Localization And Mapping). ARKit/ARCore achieve $\pm 1\text{--}3\text{cm}$ at close range via tracked visual features.

Geo-Registration

Aligning digital twin coordinate system with GPS. Sub-decimeter accuracy requires HD map matching or pre-placed visual reference targets.

Construction Site VR

BIM model overlaid on empty lot via iPad Pro + ARKit. Developer walks the lot and sees proposed 40-story tower at true scale with dynamic shadows.

Real-World Occlusion

Digital objects must be hidden when a real object (person, car) passes in front. LiDAR depth sensor (iPhone Pro, HoloLens 2) enables accurate occlusion.

Urban AR Challenges

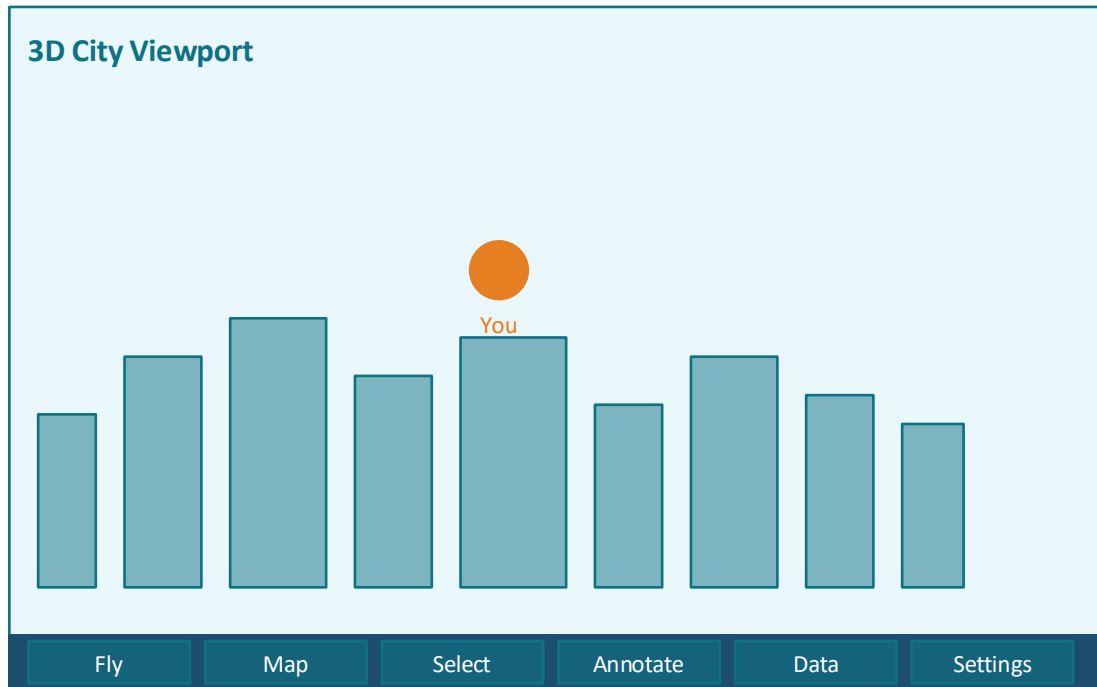
GPS multipath in dense canyons ($\pm 20\text{--}50\text{m}$ error). Rapid outdoor lighting changes impair visual SLAM. Scale of city-scale AR degrades SLAM accuracy.

Heritage & Inspection

Historical overlays on existing buildings, underground pipe visualization, as-built verification (scan vs. BIM), pavement condition assessment.

Design Patterns for Urban Dashboards

Integrating 2D charts and data filters into the 3D view.



KPI Summary

Population: 2.3 M
Buildings: 45,230 · Avg Height: 12 floors
Green Space: 18%

Traffic Flow (Live)

N-S corridor:  68%
E-W corridor:  82%
Ring road:  44%

Filter Controls

Year Built: [1900 – 2025]
Use: ☒ Office ☒ Residential ☐ Industrial
Zone: [All ▾]

Coordinated views: every action in any view (3D click, filter change) propagates to all other views via shared state store.

Evaluating 3D Interaction

User testing metrics: task completion time, error rate, SUS, cybersickness.

Metric	What It Measures	Target Threshold
Task Completion Time	Time to navigate and complete a representative planning task	< 30 s for known-location tasks
Error Rate	% incorrect selections or wrong-destination navigation events	< 5 % error rate target
SUS Score	System Usability Scale — 10-question, 5-pt Likert (scaled 0–100)	Score > 68 = above average
Presence Score (IPQ)	How 'real' does the virtual city feel? (IPQ / ITC-SOPI)	Score > 3.5 / 5.0
Cybersickness (SSQ)	Simulator Sickness Questionnaire — nausea, oculomotor, disorientation	SSQ total score < 10
Learnability	Time to first successful task with zero prior training	< 5 min for basic navigation

Lab Study

Eye tracking + think-aloud + screen recording. High internal validity.

Expert Review

Heuristic evaluation against Bowman's 3DUI guidelines (7 heuristics).

Future Trends

What is coming in the next 5–10 years for urban visualization.

Near-term (1–3 yrs)

AI-Assisted Navigation

LLM natural language queries drive 3D camera: 'Show me all pre-war office buildings near transit.' Intent parsed → spatial query executed → camera flies to results.

Near-term (now → 2 yrs)

3D Gaussian Splatting (3DGS)

Photorealistic scene representation from drone photos — no manual modeling. Trained in ~30 min. Real-time rendering on consumer GPU. Deployed: Luma AI, Polycam.

Mid-term (3–7 yrs)

Digital Twin Convergence

IoT + BIM + GIS + simulation in one unified real-time platform. Singapore Virtual Singapore, Helsinki 3D+ are proof-of-concept leaders. Schema interoperability remains the key challenge.

Long-term (7–15 yrs)

Brain-Computer Interfaces (BCI)

EEG-based intent detection + gaze tracking for 3D navigation. Meta EMG wristband, Neuralink research-phase. Near-term: gaze + implicit intent detection with consumer headsets.

Conclusion & Q&A

Summary of key performance and interaction takeaways.

Performance

LOD + Culling + Batching + Streaming = interactive city at 60 fps.

Navigation

No single metaphor covers all scales — design scale-appropriate hybrid systems.

Collaboration

Avatar + pointer + spatial audio = productive multi-user remote design review.

Spatial Data

Quadtrees/Octrees enable $O(\log n)$ queries over city-scale datasets.

Selection

Raycasting + BVH for precise pick; cone/lasso/radius for multi-object operations.

Future

AI navigation, 3D Gaussian Splatting, digital twins, and BCI reshape urban viz.

Discussion Questions

Q1: What performance technique will matter most for your capstone project — and why?

Q2: Which 3D interaction metaphor would you choose for a public urban planning tool accessible to non-expert citizens?